

HTML — Detailed Study Notes

Frontend Developer • Step 1: Learn HTML • HTML Fundamentals + Semantic HTML

How to use these notes: Read each explanation, type the examples yourself, then complete the exercises. This is intentionally much more detailed than a cheat sheet.

1. What is HTML?

HTML (HyperText Markup Language) is the standard markup language used to structure and describe content on the Web. It tells a browser what pieces of content are: headings, paragraphs, links, images, lists, forms, navigation regions, articles and so on. HTML is not a general-purpose programming language.

HTML vs CSS vs JavaScript: HTML gives structure and meaning; CSS controls presentation such as colors, spacing and layout; JavaScript adds behavior and dynamic interaction.

Example: a button is represented by HTML, styled by CSS, and its click behavior can be implemented with JavaScript.

2. How a Browser Processes HTML

```
HTML source
  |
  v
HTML parser
  |
  v
DOM tree
  |
  +----- CSS information
  |
  +----- JavaScript
  |
  v
Browser renders the page
```

The **DOM (Document Object Model)** is a structured tree representation of the HTML document. JavaScript can find elements in the DOM and modify them.

3. Elements, Tags, Attributes and Content

```
<a href="/careers" class="nav-link">Careers</a>
^      ^              ^
tag    attributes      content
```

Complete line = element

A **tag** is markup such as `<a>`. An **element** is the complete construct, including its content and tags when applicable. An **attribute** supplies additional information or configuration. Attributes normally appear in the opening tag as `name="value"`.

Common global attributes include `id`, `class`, `title`, `lang`, `hidden` and `data-*` attributes. Element-specific attributes include `href`, `src`, `alt`, `type` and `name`.

4. HTML Document Structure

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport">
```

```

        content="width=device-width, initial-scale=1.0">
    <title>CareerCompass</title>
</head>
<body>
    <h1>Frontend Developer</h1>
    <p>Learn HTML step by step.</p>
</body>
</html>

```

doctype: tells the browser to use modern HTML parsing mode. **html:** root of the document. **head:** metadata and resource information. **body:** content of the page. **title:** document title. **meta charset:** character encoding. **viewport:** mobile viewport configuration.

5. Headings and Text

```

<h1>Frontend Developer Roadmap</h1>
<h2>HTML</h2>
<h3>HTML Fundamentals</h3>
<p>HTML structures web content.</p>
<p><strong>Important:</strong> Practice every example.</p>
<p><em>Tip:</em> Type code yourself.</p>

```

HTML has six heading levels, h1 through h6. Use them to express hierarchy, not simply to make text appear large. Use CSS for visual sizing.

6. Links and Images

```

<a href="https://developer.mozilla.org/">MDN</a>

<a href="about.html">About</a>

<a href="#contact">Contact section</a>



```

An anchor's href defines its destination. A relative URL such as about.html is resolved from the current document. An image's src identifies the image resource and alt supplies a text alternative.

Accessibility rule: meaningful images need useful alternative text. Decorative images can often use alt="".

7. Lists

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<ol>
  <li>Plan</li>
  <li>Build</li>
  <li>Test</li>
</ol>

<dl>
  <dt>HTML</dt>
  <dd>Markup language for web structure.</dd>
</dl>
```

Use ul when order is not important, ol when sequence matters, and dl for term/description relationships.

8. Tables

```
<table>
  <caption>Career Skills</caption>
  <thead>
    <tr>
      <th scope="col">Skill</th>
      <th scope="col">Level</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">HTML</th>
      <td>Beginner</td>
    </tr>
  </tbody>
</table>
```

Tables are for genuinely tabular data. Do not use tables to create webpage layouts. caption names the table; thead/tbody group rows; th identifies header cells; td identifies data cells.

9. Forms and User Input

```
<form action="/register" method="post">
  <label for="name">Name</label>
  <input id="name" name="name"
    type="text" required>

  <label for="email">Email</label>
  <input id="email" name="email"
    type="email" required>

  <button type="submit">Register</button>
</form>
```

A form groups controls for collecting/submitting information. A label should be associated with its control. The name attribute identifies submitted data. required enables native browser validation.

```
<select name="role">
  <option value="frontend">Frontend Developer</option>
  <option value="backend">Backend Developer</option>
</select>

<textarea name="message"></textarea>
```

10. Semantic HTML — The Core of This Step

Semantic HTML means choosing an element because its name communicates the meaning or role of the content. This makes the document easier for people, browsers, assistive technologies and other tools to understand.

```
<header>...</header>
<nav>...</nav>
<main>...</main>
<section>...</section>
<article>...</article>
<aside>...</aside>
<footer>...</footer>
```

header: introductory content for a page or section. **nav:** important navigation links. **main:** primary content. **section:** thematic grouping, normally with a heading. **article:** self-contained content. **aside:** related/tangential content. **footer:** footer information.

11. div and span

div and span are generic containers. They are useful when you need grouping but no more meaningful semantic element fits. Do not replace every semantic element with a div.

```
<div class="card">
  <h2>HTML</h2>
  <p>Learn structure.</p>
</div>
```

```
<p>Learn <span class="highlight">HTML</span>.</p>
```

12. Building a Semantic Page

```
<body>
  <header>
    <a href="/">CareerCompass</a>
  </header>

  <nav aria-label="Main navigation">
    <a href="/careers">Careers</a>
    <a href="/roadmaps">Roadmaps</a>
  </nav>

  <main>
    <article>
      <header>
        <h1>Frontend Developer</h1>
        <p>Build modern web interfaces.</p>
      </header>

      <section>
        <h2>Roadmap</h2>
        <p>...</p>
      </section>

      <section>
        <h2>Projects</h2>
        <article>
          <h3>Portfolio Website</h3>
          <p>...</p>
        </article>
      </section>
    </article>

    <aside>
      <h2>Related Careers</h2>
      <p>UI Designer</p>
    </aside>
  </main>

  <footer>CareerCompass</footer>
</body>
```

This structure communicates the page's major regions without requiring CSS or JavaScript to explain what they are.

13. Accessibility Essentials

Semantic HTML is an accessibility foundation. Use a logical heading hierarchy; give meaningful images useful alt text; label form controls; use links for navigation and buttons for actions; and preserve keyboard operability.

Button vs link: if the user is going somewhere, use an anchor. If the user performs an action on the current page, use a button.

ARIA: use native HTML first. ARIA can supplement semantics when native HTML cannot express the required behavior, but adding ARIA unnecessarily can make accessibility harder.

14. Media and Interactive HTML

```
<video controls width="640">
  <source src="lesson.mp4" type="video/mp4">
</video>

<audio controls src="lesson.mp3"></audio>

<details>
  <summary>What is HTML?</summary>
```

```
<p>HTML structures web content.</p>
</details>
```

For educational video/audio, provide appropriate captions or transcripts where possible. details/summary provides a native disclosure pattern.

15. Void Elements

Void elements do not wrap child content and do not use closing tags. Common examples are img, input, meta, link, br and hr.

```

<input type="email">
<meta charset="UTF-8">
<br>
```

16. Nesting and Common Errors

```
<!-- Correct -->
<p>This is <strong>important</strong>.</p>
```

```
<!-- Incorrect -->
<p>This is <strong>important.</p></strong>
```

Correct nesting matters because HTML is parsed into a tree. Incorrectly nested elements can produce unexpected structures and accessibility issues.

Other common mistakes: using headings for visual size, using `br` for spacing, using tables for layout, missing form labels, using clickable divs, and placing secrets in HTML source/comments.

17. HTML Entities and Comments

```
<!-- developer comment -->

<p>Use &lt;h1> for a main heading.</p>
<p>Tom &amp; Jerry</p>
<p>She said &quot;Hello&quot;.</p>
```

Comments are ignored as normal page content but are visible in source. Character references represent special characters that have meaning in HTML.

18. File Paths

```
project/
■■■ index.html
■■■ about.html
■■■ images/
■   ■■■ logo.png
■■■ pages/
    ■■■ contact.html

<!-- index.html -->


<!-- pages/contact.html -->

```

A relative path is resolved from the current document's location. `../` moves one directory upward.

19. Mini Project — Career Roadmap

Create an HTML-only page with:

- semantic header and navigation
- main career introduction
- roadmap ordered list
- project articles
- related-career aside
- accessible images
- FAQ using details/summary
- contact form
- footer

Do not use CSS at first.
The goal is correct structure and semantics.

20. Self-Test Questions

1. What does HTML stand for?
2. Why is HTML not considered a programming language?
3. What is an element?

4. What is an attribute?
5. What does the DOM represent?
6. Why is alt text important?
7. ul vs ol?
8. Why should tables not be used for layout?
9. section vs article?
10. When should you use nav?
11. div vs semantic elements?
12. button vs anchor?
13. Why are labels important?
14. What are void elements?
15. Why does correct nesting matter?

21. Final Checklist

- I can create an HTML document from memory.
- I understand tags, elements and attributes.
- I can create links, images, lists and tables.
- I can create a labeled form.
- I understand semantic HTML.
- I can choose header/nav/main/section/article/aside/footer correctly.
- I understand div and span.
- I know basic accessibility practices.
- I can build a complete HTML-only page without copying.

Technical references used while preparing these notes: MDN's current HTML fundamentals and Semantic HTML curriculum. MDN's material covers syntax, document structure, links, lists, images, forms, media, tables, debugging and the importance of machine-readable semantics for accessibility and SEO.